

Thesis Report

The use of generative deep learning to synthesize realistic healthcare digital twins

Laura Jochim

1. Introduction

A recent development within the research field of medicine is the creation of a more holistic approach and the targeting of person-specific treatments. One relatively new approach to achieving a holistic medical research field is the creation of digital twins. A digital twin is a virtual replica of a real human patient [1]. Digital twins can help clinicians to gain valuable insights, optimize treatment strategies, and deliver personalized care. Furthermore, the application of digital twins can help reduce animal testing, ensure privacy when sharing data, and address ethical concerns during data collection [2]. Even though the use of digital twins within the healthcare sector is a relatively new development, it is known to be a promising tool to virtually replicate human body parts or create data-driven digital twins [3], [4].

Such a digital twin is created on the basis of synthetic data, i.e., fake data that resembles real-world data. The idea of synthetic data was initially proposed by Rubin [5], where he suggested drawing synthetic samples using a multiple imputation approach to handle missing data [6], [7], similar to what is implemented in R by mice [8]. In the following years, advanced, machine learning, techniques have been incorporated to account for complex sampling

December 2024

designs [9]. The computer science field also started to develop approaches, many were developed around 2014. Examples are Bayesian networks [10], copulas [11], and generative adversarial networks (GANs) [12]. Especially GANs resulted in a boost in synthetic data research and applications in the computer science literature [9].

This project will focus on GANs to synthesize digital twins. GANs have been widely used in several fields to generate images and sound data. In healthcare, a recent increase in the popularity of GANs has been seen, showing good performance in literature especially because of their flexibility [13]. Briefly, a GAN is a neural network involving a generator and a discriminator [12]. The generator produces synthetic samples and is finetuned through the discriminator to generate data matching the training data characteristics. An example of how the GAN generates data that increasingly resembles training data is shown in Figure 1. GANs are usually used for image data, as seen in Figure 1, though healthcare data is often represented in a table, where rows represent the patient and columns the patient’s characteristics. The problems GANs often face when dealing with this kind of tabular data include (1) simultaneously modeling continuous and discrete variables, (2) the columns with complicated distributions, often a mixture of non-Gaussian distributions, and (3) the severe imbalance of categories within a column (over- or underrepresentation of a category) [15]. To deal with these issues the conditional tabular GAN (ctGAN) [16] has been developed which will be used for this project.

Another characteristic of healthcare data is that it usually has a rather small sample size, which could lead to problems. Deep learning methods like GANs

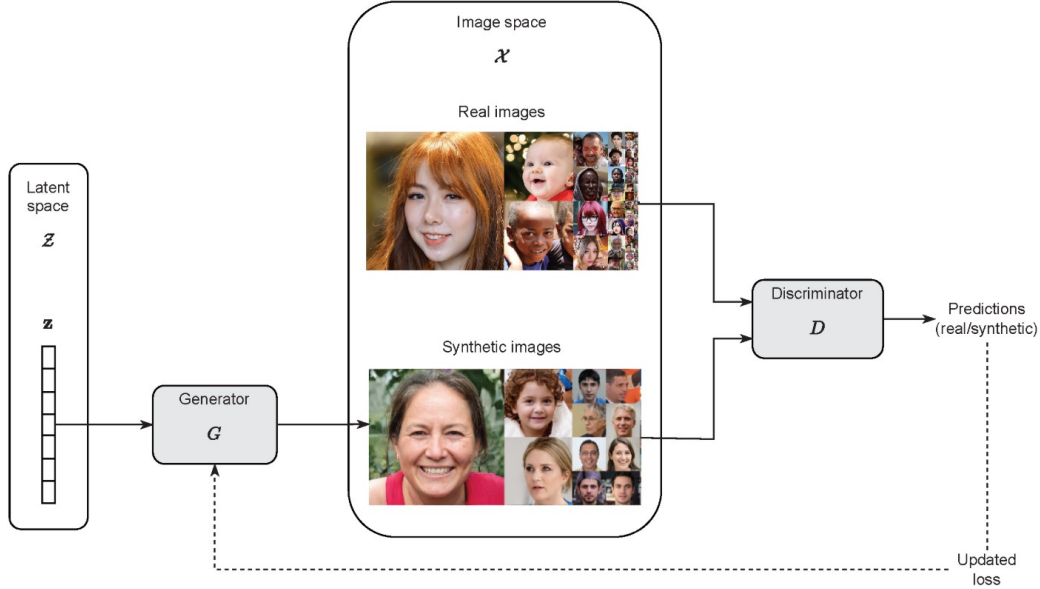


Figure 1: A visual example of how a GAN works with imaging data [14]. The discriminator is presented with both real and synthetic images of faces where the synthetic images are generated from a latent space Z . The discriminator’s decision (real/fake) is used to update the loss of the generator which in return tries to produce more realistic looking images iteratively.

typically require a large sample size to train on to prevent over- and under-sampling of specific data characteristics. GANs may also be prone to overfit when only a small sample size is available. Efforts have been made to include prior knowledge in the training phase to reduce overfitting [17]. Therefore, the aim is to implement prior knowledge in the flexible ctGAN structure and evaluate whether this works within the healthcare field. The research questions of this project are: To what extent are ctGANs applicable given the challenges of small sample-sized healthcare data? Specifically, will including prior knowledge in the ctGAN framework lead to more realistic digital twins even when the sample size is limited?

2. Methodology

The workflow of this paper is based on the ctGAN architecture [18] with modifications necessary to deal with healthcare data. In this section, we will give an overview of what a neural network is, how a GAN works, and how the ctGAN architecture handles issues encountered with healthcare data, that the GAN architecture struggles with. Lastly, we will elaborate on how to evaluate the ctGAN’s performance.

2.1. Neural Networks

A GAN consists of two neural networks, so it is important to introduce what a neural network is. A neural network is a non-linear mapping that consists of one or more layers of latent (or hidden) variables. The input layer consists of the data variables while the subsequent layers consist of variables that are non-linear transformations of the output of the previous layer. These transformations are generally constructed by a linear transformation, including an intercept, of the previous variables and a non-linear activation function. For example, a particular latent variable in a hidden layer can be written as $f(x) = \varphi(x^\top w + b)$. Some often chosen activation functions are a rectified linear unit (RELU) and a sigmoid. The output y is then modeled in terms of x via a chain of activation functions and linear transformations, e.g. $y = f_1(f_2(f_3(x)))$ for three hidden layers. The parameters that are estimated during training are the weights w and intercepts b from each layer. To optimize these parameters, numerical optimization techniques are used that are implemented in Python deep learning libraries such as PyTorch [19].

2.2. Generative Adversarial Network

The GAN framework, introduced by Ian Goodfellow et al. [12], consists of two competing neural networks: a generator and a discriminator, which are trained simultaneously in a game-theoretic setup; see Figure 1.

The generator transforms samples from random input z sampled from a prior distribution ($p_z(z)$), typically uniform or Gaussian, to a data space aiming to obtain realistic data samples by capturing the data distribution. The discriminator takes an input x and outputs y , which is a binary outcome, indicating whether the input comes from generated or real data ($p_{data}(x)$). Then the discriminator’s evaluation (“fake”/“real”) is used to compute the generator’s loss, which is then minimized through backpropagation, updating the generator’s weights to improve its performance.

The idea is to train the generator enough iterations that it starts producing output resembling the training data and ‘fool’ the discriminator into classifying the generated data as real. In theory, the training is completed when the discriminator cannot tell generated and real data apart and therefore has a probability of 0.5, i.e., random classification. This minmax game can be written as a value function,

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - D(G(z)))], \quad (1)$$

where G annotates the generator and D the discriminator. The first term trains the discriminator to maximize the probability of identifying the real

data, ideally close to 1, whereas the second term trains the discriminator to minimize the probability of identifying the fake data from the generator as real, ideally close to 0. Note that for a well-trained discriminator, $D(x)$ is close to 1 and $D(G(z))$ is close to zero, maximizing the combined score V . And for a well-trained generator, the $D(G(z))$ is close to 1, which worsens the performance of the discriminator. Once convergence is reached and the GAN is trained to produce real-looking data, one can use the trained generator to obtain synthesized data.

2.3. CTGAN

For this project it is especially interesting to look at the ctGAN architecture, a GAN designed for tabular data, which has been shown to outperform alternative architectures for tabular data [15]. The GAN structure is most often used for image data and is therefore useful in e.g., medical imaging. This project will deal with medical tabular data, where each patient is listed in a row with columns representing clinical data e.g., patient demographic, information about mutated genes, and cancer tumor stage. The problems GANs usually run into when dealing with tabular data were addressed using the ctGAN [16], see Figure 2.

To overcome the issue of modeling non-Gaussian and multimodal distribution, i.e. columns with complicated distribution, the developers of ctGAN invented mode-specific normalization [15]. In this method, each column is processed independently in three steps. For each column, the number of modes is estimated and a Gaussian mixture [21] is fitted. Then for each value in the column, its membership probability for each of the mixture

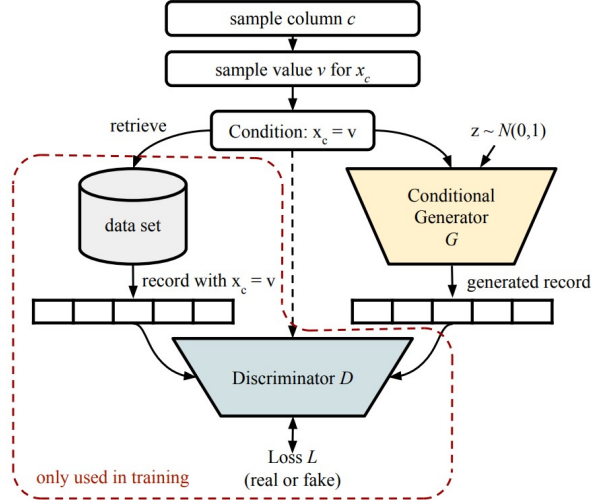


Figure 2: For the ctGAN [20], the generator has a condition term. First, a column and then a value of said column is sampled. This condition $x_c = v$ and noise $z \sim \mathcal{N}(0,1)$ is fed into the generator G to produce a synthetic record that adheres to the condition. Both the real record (retrieved from the dataset) and the generated record are passed to the Discriminator D . The Loss L , calculated based on the discriminator’s output, is used to update both the generator and discriminator during training.

distributions is computed. Lastly, the values are then z-transformed with respect to one of the mixture distributions using the estimated mean and variance of that Gaussian distribution. Note, that to capture the full diversity of the data’s distribution, mode-specific normalization is dynamically applied throughout the training process. Instead of the usual normalizing in the preprocessing stage, the generator can adapt to the data’s structure and conditional dependencies throughout the training.

To address the issue of imbalanced categorical variables ctGAN implements a conditional generator and training-by-sampling. The generator is conditioned to focus on specific categories, mainly less represented categories, during the learning process to be able to create more realistic data even when training data is limited. The loss function includes a penalty for the

generator when its output does not match the desired category. This penalty ensures that the generator learns to produce data that aligns with the given conditions, making its outputs more accurate.

At the same time, training-by-sampling ensures that the final data reflects the real distribution by avoiding bias through over- or undersampling. This is done by sampling based on the log-frequency of each category. It ensures that less frequent categories are given more representation during training without distorting the overall data distribution.

To incorporate prior knowledge in the ctGAN architecture a regularization term $R(G)$ is introduced which is weighted by the hyperparameter λ modifying the loss functions as follows:

$$\min_G \max_D (V(G, D + \lambda R(G))) \quad (2)$$

The loss function governs how both components, generator and discriminator, improve over time. Including a regularization term aids in implementing prior knowledge by introducing constraints. A penalty can be imposed to reflect the prior knowledge that certain features should (not) be correlated. The neural network architecture underlying the generator G will then be penalized towards (or away from) inducing correlations between these features. Following the three-stage method developed by Anderson, Gadgil, Johnson, Schwab, and Davidson (2021) [22], the inclusion of prior knowledge can be achieved by combining it with the output of the final layer of the neural

network. In the thesis, we will further explore possible ways to incorporate prior knowledge where available.

2.4. Data and Software Description

The data used for this project is the following freely accessible and comprehensive datasets: the TCGA Breast Cancer (BRCA) dataset from cBioPortal [23] and the 10,000-Patient Database from Figshare [24]. The BRCA dataset, with a sample size of 1101, provides detailed clinical and molecular data for breast cancer patients. We consider demographics, treatment history, cancer subtypes, proteomics, and genomics data.

The 10,000-Patient Database contains clinical and demographic information on virtual patients across various healthcare settings, with variables such as patient admission details, diagnoses, demographics, socioeconomic details, and approximately 50 diagnostic lab measurements, across multiple time points.

An interface to ctGAN is implemented in R and Python. We use the R package RCTGAN [25] for the first exploration stages in the software RStudio [26]. Incorporating prior knowledge is performed in Pytorch [19] either by implementing a custom generator and discriminator, or using the Python-CTGAN library [27].

Table 1: A virtual patient repository of 10,000 patients.

Variable and category	Patients (n = 10,000)
Mean age as of 1/1/2015, years (SD)	57.8 (17.3)
Gender (%)	
Female	52.0
Ethnicity (%)	
White	49.0
Asian	23.0
African American	15.0
Unknown	13.0
Mean number of admissions per patient (SD)	3.6 (1.5)
Mean length of stay, days (SD)	11.0 (5.2)
% Population with length of follow-up (years)	
0–9	13.1
10–15	9.3
>15	77.6
Population below poverty (%)	21.6
Comorbidities; Prevalence (%)	
Malignant neoplasm	41.4
Rheumatoid arthritis	25.6
Diabetes (type I or II)	24.4
Renal complications	17.0
Coronary artery disease	7.0
Laboratory values (Mean; SD)	
Blood urea nitrogen (mg/dL)	17.5; 7.2
Platelets (k/cumm)	284.9; 95.3
Creatinine (mg/dL)	0.9; 0.2
Albumin (gm/dL)	4.2; 1.0
Lymphocytes (%)	25; 5.8

2.5. Analysis plan

To evaluate the GAN’s performance the training-test-split procedure is applied. The common 80/20 split is used where the GAN is trained on 80

percent of the data. In the process of training a GAN different parameters are tuned, like the learning rate and the optimizer which will be explored during the project. Once convergence is reached, the generator is used to synthesize data.

To evaluate this newly obtained data on how realistic it is, the synthetic data is compared to the 20 percent test data. First, the distributions are compared on means, variances, skewness, and kurtosis. A permutation test is performed to test whether these quantities are different between real and synthetic data. Then we look at whether the correlations present in the real data are also present in the synthetic data. Here we plot a heatmap of the correlation to see where correlations tend to overlap. A few key variables are chosen to see whether their correlation is similar in the real and synthesized data. Lastly, the Kullback-Leibler divergence, which is the expected value of the log-likelihood ratio, is used to identify how dissimilar the probability distributions of the synthetic and real data are. A Kolmogorov-Smirnov test is then conducted to investigate whether the difference between the probability distributions of synthetic and real data is significantly different. Based on the information obtained, we decide whether the GAN is trained properly and performs well in synthesizing data.

References

- [1] C. Meijer, H.-W. Uh, and S. el Bouhaddani, “Digital twins in healthcare: Methodological challenges and opportunities,” *Journal of Personalized Medicine*, vol. 13, no. 10, 2023.
- [2] K. Subramanian, “Digital twin for drug discovery and development—the virtual liver,” *Journal of the Indian Institute of Science*, vol. 100, pp. 653 – 662, 2020.
- [3] K. Gillette, M. A. Gsell, A. J. Prassl, E. Karabelas, U. Reiter, G. Reiter, T. Grandits, C. Payer, D. Štern, M. Urschler, J. D. Bayer, C. M. Augustin, A. Neic, T. Pock, E. J. Vigmond, and G. Plank, “A framework for the generation of digital twins of cardiac electrophysiology from clinical 12-leads ecgs,” *Medical Image Analysis*, vol. 71, p. 102080, 2021.
- [4] B. Kovatchev, “A century of diabetes technology: Signals, models, and artificial pancreas control,” *Trends in Endocrinology Metabolism*, vol. 30, no. 7, pp. 432–444, 2019.
- [5] Donald B. Ruben, “Statistical disclosure limitation,” Tech. Rep. 2, Statistics Sweden, 1993. Accessed: 2024-06-17.
- [6] P. R. Rosenbaum and D. B. Rubin, “The central role of the propensity score in observational studies for causal effects,” *Biometrika*, vol. 70, no. 1, pp. 41–55, 1983.
- [7] D. B. Rubin, “Multiple imputations in sample surveys—a phenomenological bayesian approach to nonresponse,” 2002.

- [8] S. van Buuren and K. Groothuis-Oudshoorn, “mice: Multivariate imputation by chained equations in r,” *Journal of Statistical Software*, vol. 45, no. 3, pp. 1–67, 2011.
- [9] J. Drechsler and A.-C. Haensch, “30 Years of Synthetic Data,” *Statistical Science*, vol. 39, no. 2, pp. 221 – 242, 2024.
- [10] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, “Privbayes: private data release via bayesian networks,” in *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, SIGMOD ’14, (New York, NY, USA), p. 1423–1434, Association for Computing Machinery, 2014.
- [11] H. Li, L. Xiong, and X. Jiang, “Differentially private synthesization of multi-dimensional data using copula functions,” *Advances in database technology : proceedings. International Conference on Extending Database Technology*, vol. 2014, pp. 475–486, 11 2014.
- [12] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial networks,” 2014.
- [13] A. Aggarwal, M. Mittal, and G. Battineni, “Generative adversarial network: An overview of theory and applications,” *International Journal of Information Management Data Insights*, vol. 1, no. 1, p. 100004, 2021.
- [14] C. Pasquini, F. Laiti, D. Lobba, G. Ambrosi, G. Boato, and F. De Natale, “Identifying synthetic faces through gan inversion and biometric traits analysis,” *Applied Sciences*, vol. 13, no. 2, 2023.

- [15] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling tabular data using conditional gan,” 2019.
- [16] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling tabular data using conditional GAN,” *CoRR*, vol. abs/1907.00503, 2019.
- [17] K. T. Ahmed, J. Sun, S. Cheng, J. Yong, and W. Zhang, “Multi-omics data integration by generative adversarial network,” *Bioinformatics*, vol. 38, pp. 179–186, 08 2021.
- [18] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling tabular data using conditional gan,” in *Advances in Neural Information Processing Systems*, 2019.
- [19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, Curran Associates, Inc., 2019.
- [20] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, “Deep neural networks and tabular data: A survey,” *CoRR*, vol. abs/2110.01889, 2021.
- [21] C. Bishop, *Pattern Recognition and Machine Learning*. Springer, January 2006.

- [22] P. Anderson, R. Gadgil, W. A. Johnson, E. Schwab, and J. M. Davidson, “Reducing variability of breast cancer subtype predictors by grounding deep learning models in prior knowledge,” *Computers in Biology and Medicine*, vol. 138, p. 104850, 2021.
- [23] W. Lingle, B. J. Erickson, M. L. Zuley, R. Jarosz, E. Bonaccio, J. Filip-pini, J. M. Net, L. Levi, E. A. Morris, G. G. Figler, P. Elnajjar, S. Kirk, Y. Lee, M. Giger, and N. Gruszauskas, “The cancer genome atlas breast invasive carcinoma collection (tcga-brca),” 2016. Data set.
- [24] U. Kartoun, “EMRBots: a 10,000-patient database,” 9 2018.
- [25] B. Kasaai, “ctgan: Conditional tabular gan implementation in r.” <https://github.com/kasaai/ctgan>, 2020. R package, version 0.1.0.
- [26] RStudio Team, *RStudio: Integrated Development Environment for R*. RStudio, PBC., Boston, MA, 2024.
- [27] N. Patki, R. Wedge, and K. Veeramachaneni, “The synthetic data vault,” in *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, pp. 399–410, Oct 2016.